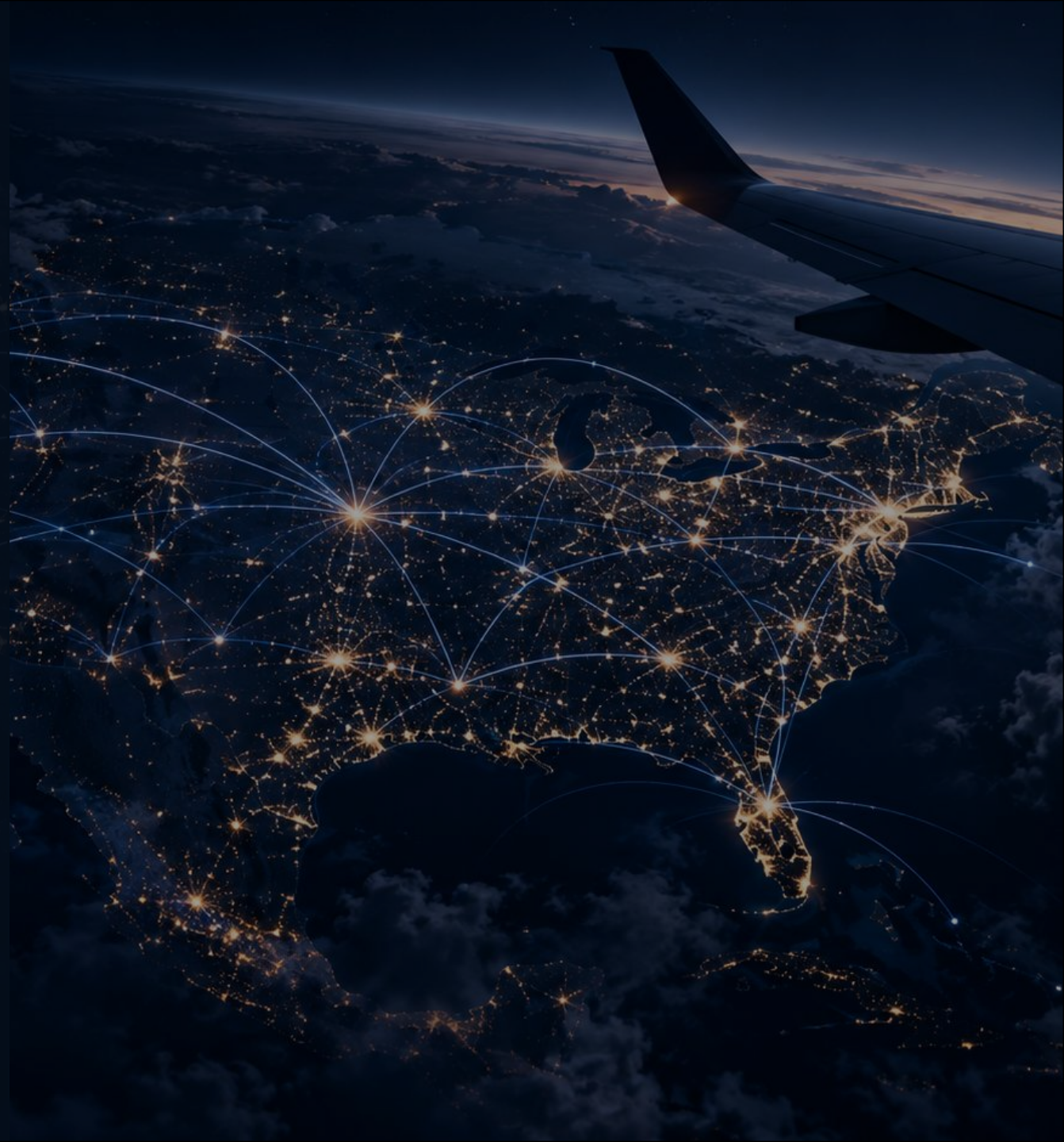
PROJET DE DATA ENGINEERING

Analyse des retards des compagnies aériennes américaines

Janvier 2019 — Juin 2026

PySpark • Python • SQL Server • Airflow • Architecture médaillon • Power BI



Kevin Issa · Anthony Quinet

Digitalcity Brussels

VUE D'ENSEMBLE

Sommaire

01 Contexte

Objectif, périmètre et questions business

02 Sources

Flights, Passengers, Runways et données météo

03 Architecture

Pipeline, architecture médaillon et technologies utilisées

04 ETL & modèle

Silver, Modèle dimensionnel et difficultés rencontrées

05 Monitoring

Durées, mémoire et volumes chargés

06 Résultats

Classement, limites et améliorations

Nos parcours

Kevin Issa

Formation

Bachelier En Sciences Informatique (ULB)

Spécialisation

Certification Data Engineer Databricks / Azure

LinkedIn

https://www.linkedin.com/in/kevin-issa-809b32385/

Anthony Quinet

Formation

Master en Droit (UCL) et en Gestion (ICHEC)

Expériences

Consultant en Gestion - Business Analyst et Project & Change Management

LinkedIn

https://www.linkedin.com/in/quinet-anthony/

Présentation du projet

Construire un pipeline de données complet pour analyser les retards des vols domestiques américains entre Janvier 2019 et juin 2026, ainsi que leurs causes.

2019–2026
Période analysée

4
Sources de données

~50 M
Lignes dans le Bronze

101
Fichiers csv



Questions business

01 Comment les retards évoluent-ils au fil des mois et des années, et quelles en sont les principales causes ?

02 Le volume de passagers, la saison, les intempéries ou l'heure de départ influencent-ils les retards ?

03 Quelles compagnies présentent les taux de retard les plus élevés ?

Sources de données

Flights

Tous les vols planifiés aux États-Unis entre 2019 et juin 2026.

Source

Bureau of Transportation Statistics

Granularité

1 ligne = 1 vol planifié

Passengers

Tous les passagers transportés sur la même période.

Source

Bureau of Transportation Statistics

Granularité

1 ligne = 1 trajet mensuel entre 2 villes

Runways

Tous les aéroports américains et les données de leurs pistes.

Source

U.S. Department of Transportation

Granularité

1 ligne = 1 piste d’aéroport

Weather API

Données météorologiques historiques.

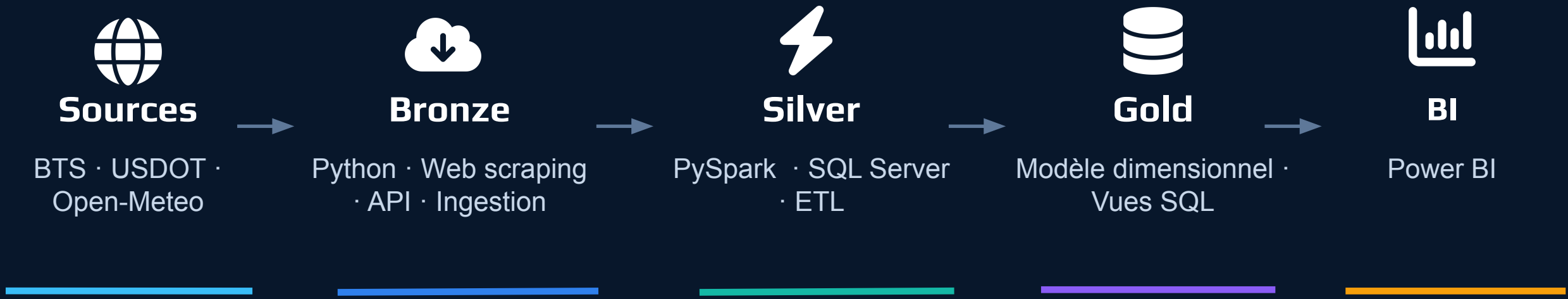
Source

Open-Meteo API

Granularité

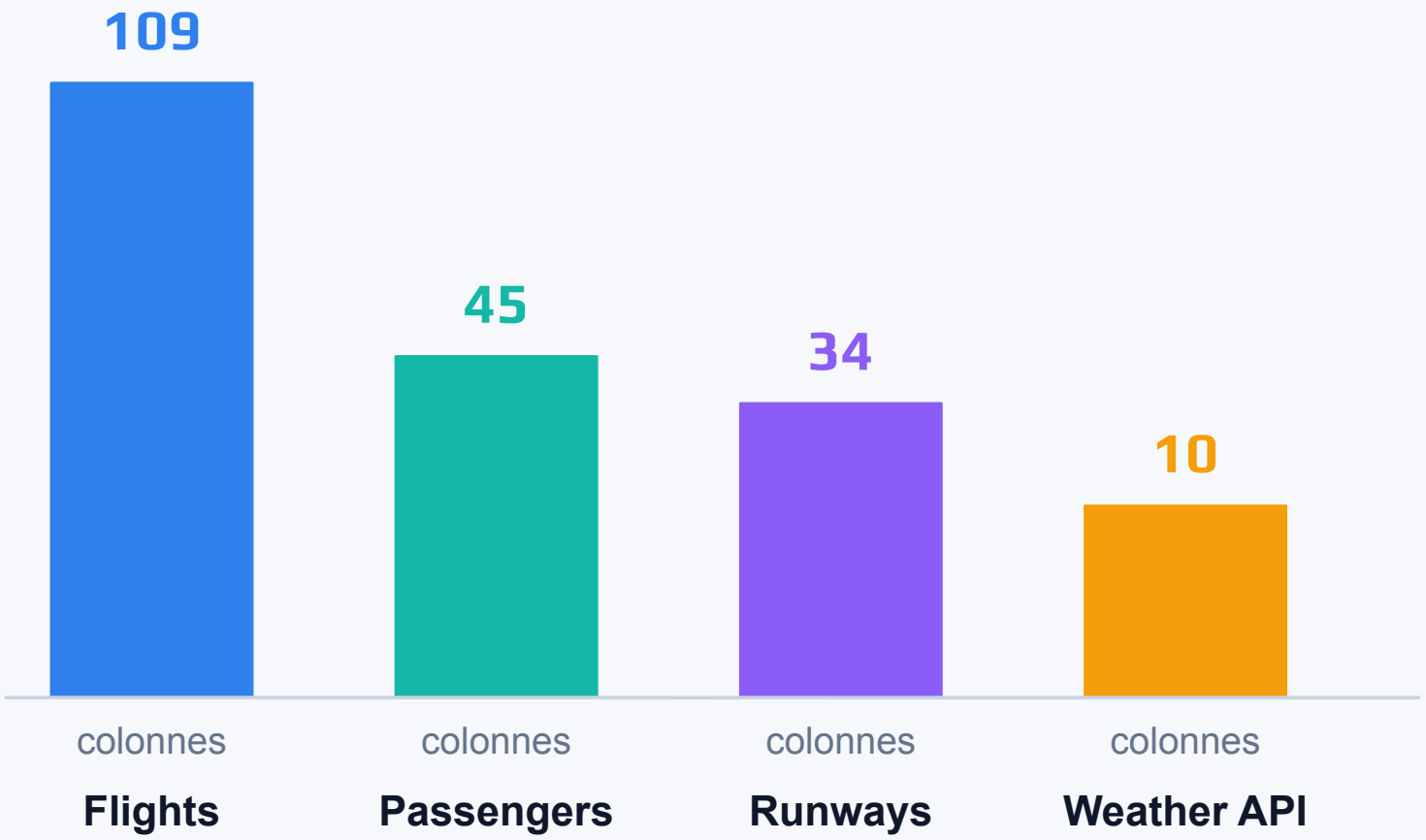
1 ligne = 1 heure d’observation météo

Architecture & technologies



Airflow orchestre les pipelines, les relances et les temps d’exécution

Bronze — volume & périmètre



≈ 50 millions

de lignes au total dans le Bronze

Périmètre analytique

Les analyses sont limitées aux compagnies présentes dans Flights afin de conserver un périmètre cohérent entre les deux tables de faits.

TempSilver — modèle d'intégration

Toutes les colonnes conservées après nettoyage et normalisation.

SegmentPassengers

SEGMENT_PASSENGERS_ID
YEAR
MONTH
QUARTER
DEPARTURES_SCHEDULED
DEPARTURES_PERFORMED
PAYLOAD
SEATS
PASSENGERS
FREIGHT
MAIL
DISTANCE
AIR_TIME
UNIQUE_CARRIER
AIRLINE_ID
CARRIER_NAME
ORIGIN_AIRPORT_ID
ORIGIN
ORIGIN_CITY_NAME
ORIGIN_STATE_ABR
ORIGIN_STATE_NM
DEST_AIRPORT_ID
DEST
DEST_CITY_NAME
DEST_STATE_ABR
DEST_STATE_NM
DISTANCE_GROUP
ROW_HASH

Weather

CALL_API_ID
HOUR_ID
WEATHER_DATE
TEMPERATURE_2M
RAIN
SNOWFALL_CM
SNOW_DEPTH_M
WIND_SPEED_10M
WIND_GUSTS_10M
LATITUDE
LONGITUDE
ROW_HASH

Runways

OID
STATE_CODE
ARPT_ID
ARPT_NAME
CITY
COUNTRY_CODE
LATITUDE
LONGITUDE
ROW_HASH

Flights

FLIGHT_ID
YEAR
MONTH
QUARTER
DAY_OF_MONTH
DAY_OF_WEEK
FLIGHT_DATE
REPORTING_AIRLINE
DOT_ID_REPORTING_AIRLINE
ORIGIN_AIRPORT_ID
ORIGIN
ORIGIN_CITY_NAME
ORIGIN_STATE
DEST_AIRPORT_ID
DEST
DEST_CITY_NAME
DEST_STATE
CRS_DEP_TIME
DEP_TIME
DEP_DELAY
TAXI_OUT
WHEEL_OFF
WHEEL_ON
TAXI_IN
CRS_ARR_TIME
ARR_TIME
ARR_DELAY
CANCELLED
CANCELLATION_CODE
CRS_ELAPSED_TIME
ACTUAL_ELAPSED_TIME
AIR_TIME
DISTANCE
DISTANCE_GROUP
CARRIER_DELAY
WEATHER_DELAY
NAS_DELAY
SECURITY_DELAY
LATE_AIRCRAFT_DELAY
DIV_AIRPORT_LANDINGS
DIV_REACHED_DEST
DIV_ACTUAL_ELAPSED_TIME
DIV_ARR_DELAY
DIV_DISTANCE
LATITUDE
LONGITUDE
ROW_HASH

Bronze → Silver — règles de préparation

Flights

- Conserver les vols annulés
- Supprimer les lignes où les informations de retard et de durée sont simultanément incomplètes.

Passengers

- Conserver uniquement les trajets avec un nombre de passagers supérieur à 0
- Limiter le périmètre aux compagnies présentes dans Flights.

Runways

- Conserver une seule piste représentative par aéroport
- Granularité suffisante pour les besoins du projet.

```
first_condition = F.col("DEP_DELAY").isNull() | F.col("ARR_DELAY").isNull()
second_condition = F.col("CRS_ELAPSED_TIME").isNull() | F.col("ACTUAL_ELAPSED_TIME").isNull()
third_condition = F.col("CANCELLED") == 1.00

df_flights = df_flights.filter(third_condition | ~(first_condition & second_condition))
```

Idempotence & contrôle des doublons

1 

Valeurs de la ligne

Toutes les colonnes métier, hors identifiant technique

2 

ROW_HASH

Empreinte unique calculée dans PySpark

3 

Index unique

Contrôle natif dans SQL Server

4 

Chargement incrémental

Les lignes déjà présentes sont ignorées

```sql
CREATE UNIQUE INDEX UX_Flights_Hash
ON dbo.Flights (ROW_HASH)
WITH (IGNORE_DUP_KEY = ON);
```

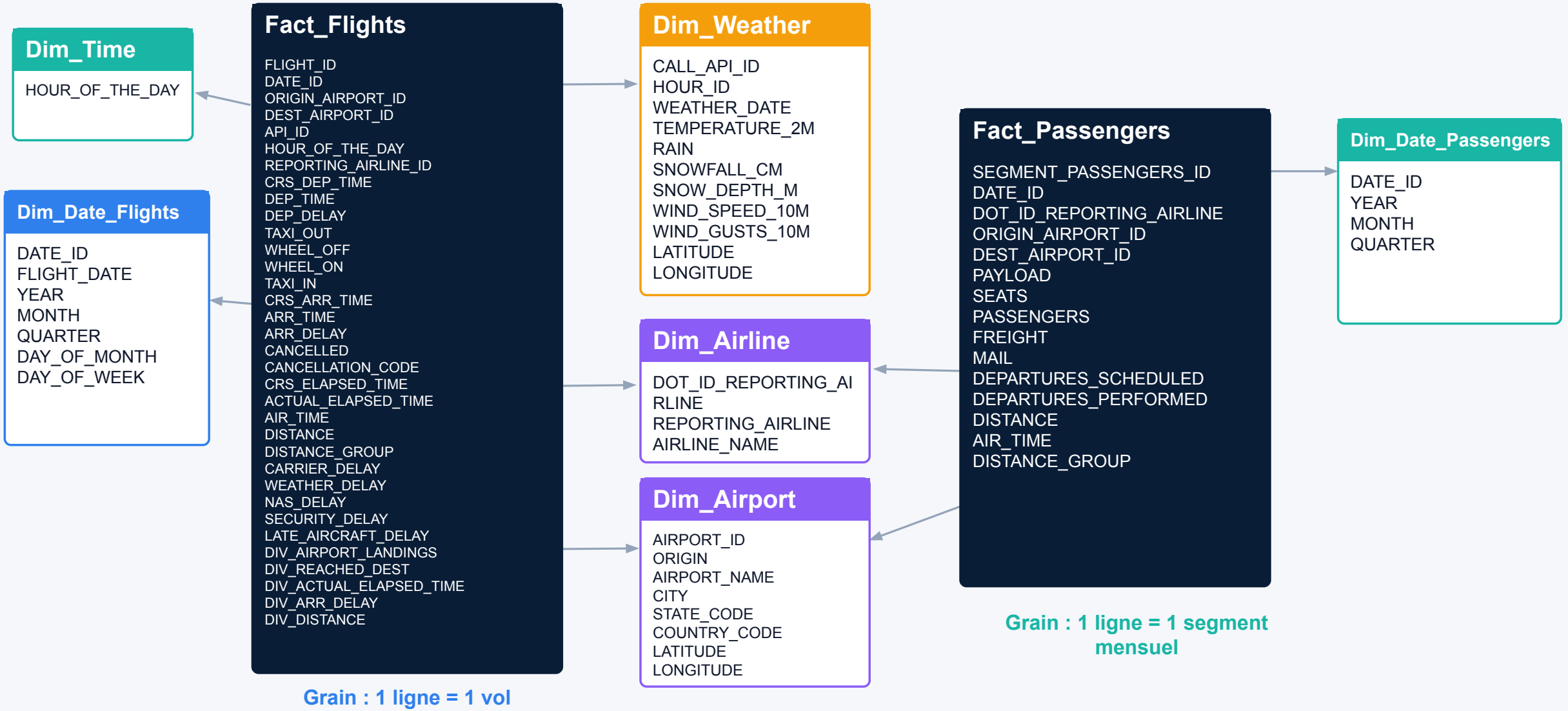
Avantages

- Relances sûres
- Déduplication efficace
- Automatisation facilitée

Coûts

- Calcul du hash
- Création et maintenance de l’index

Gold / DWH — modèle dimensionnel



ETL — problèmes rencontrés & corrections

Enrichissement météo

- Architecture et script prêts pour les calls API
- Limite de 10 000 appels par jour

Harmonisation des codes aéroport

- Code d'identification différent entre Flights et Runways
- Correspondances manuelle nécessaire pour plusieurs aéroport

AZA → IWA | FCA → GPI | HHH → HXD | YUM → NYL

Indicateur	Avant corrections	Après corrections
Passagers conservés	5 264 085 470	5 301 950 545
Part des passagers conservés	99,285 %	99,9992 %
Passagers exclus	37 908 220	43 145
Part des passagers exclus	0,715 %	0,0008 %
Trajets conservés	1 952 717	1 970 347
Part des Trajets conservés	99,10 %	99,9938 %

37,9 M

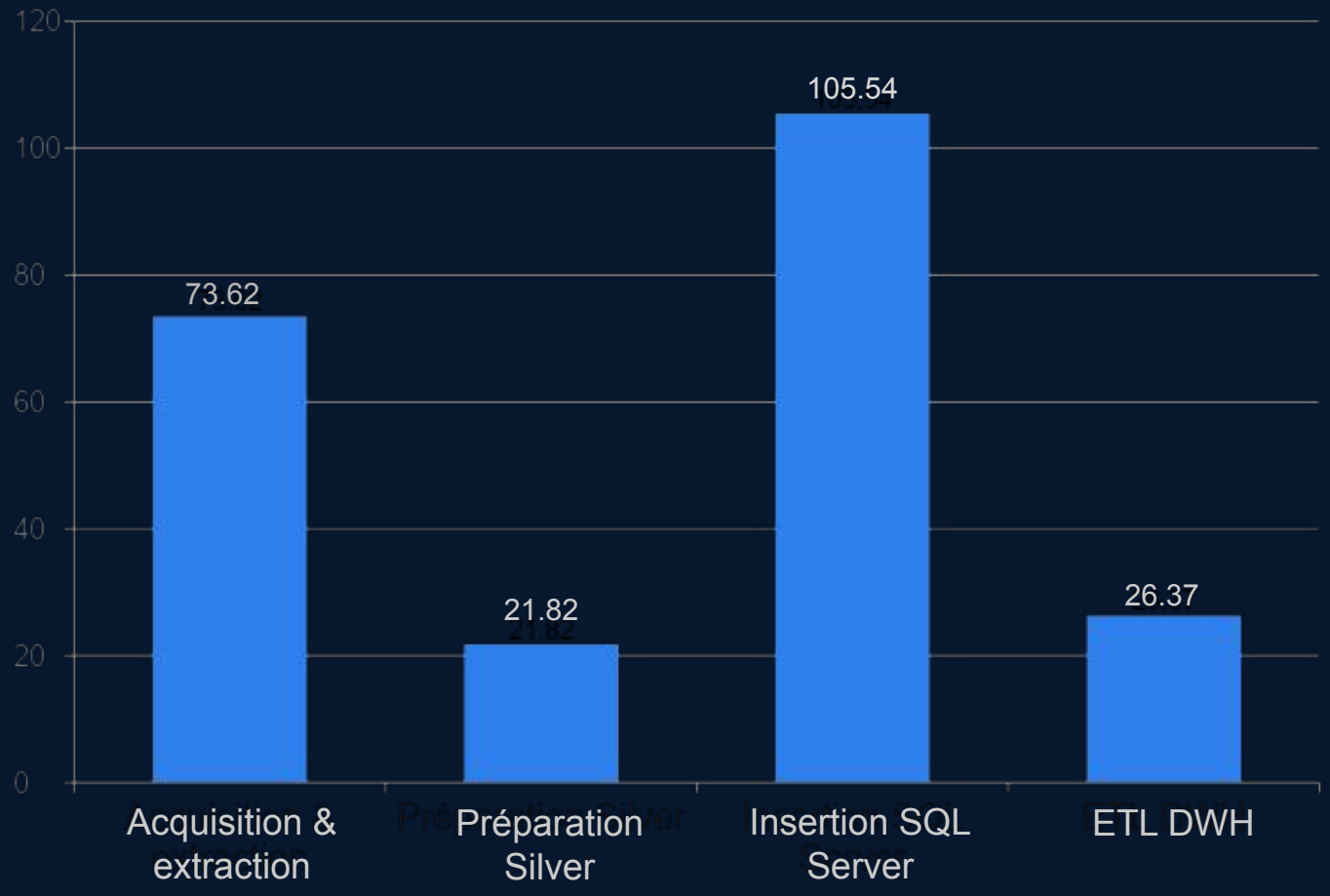
passagers exclus

↓

43 145

après harmonisation

Monitoring — temps d'exécution



Durées additionnées en minutes pour chaque bloc ; elles ne représentent pas nécessairement un temps absolu si certaines tâches sont parallélisées.

Somme des durées Airflow observées

3 h 47 min 21 s

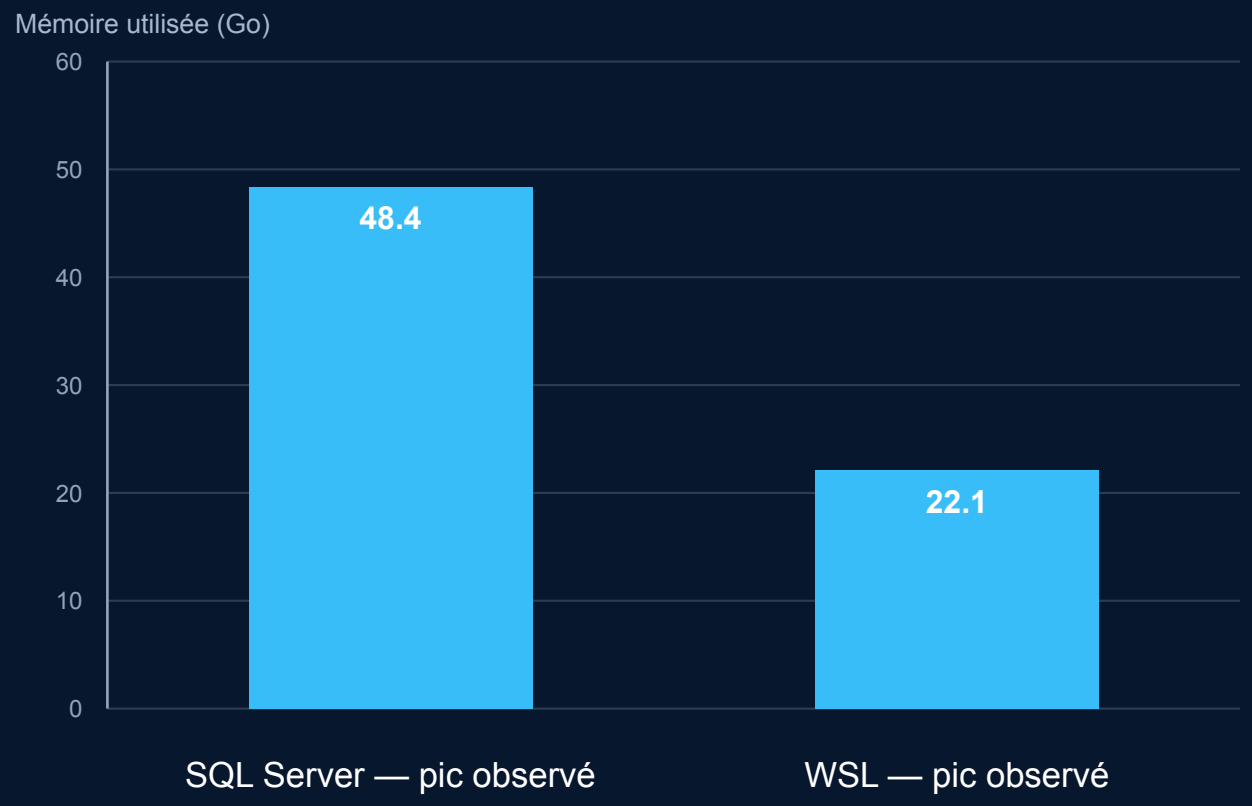
Tâches les plus longues

insert_flights_to_sql	**01:39:50**
web_scraping	**01:09:49**
load_fact_tables	**00:24:54**
prepare_flights_silver	**00:18:58**

OBSERVATIONS

Monitoring — utilisation mémoire

Captures ponctuelles réalisées pendant les tâches les plus lourdes



Lecture des captures

• La préparation des données sollicite surtout WSL / PySpark.
• Les insertions massives et les faits sollicitent surtout SQL Server.
• Les mesures sont des instantanés du Gestionnaire des tâches, pas un suivi continu.

Autres points de mesures

SQL Server : 19,0 GB (insertions) · 32,5 GB (dimensions)
WSL : 5,8 GB (insertions) · 3,2 GB (dimensions)

OBSERVATIONS

Monitoring — lignes insérées

Les logs Airflow comparent le nombre de lignes avant et après chaque tâche.

TempSilver

47 494 086	2 461 668	6 693
Flights	Passengers	Runways

Data Warehouse — faits

47 494 086	1 970 347
Fact_Flights	Fact_Passengers

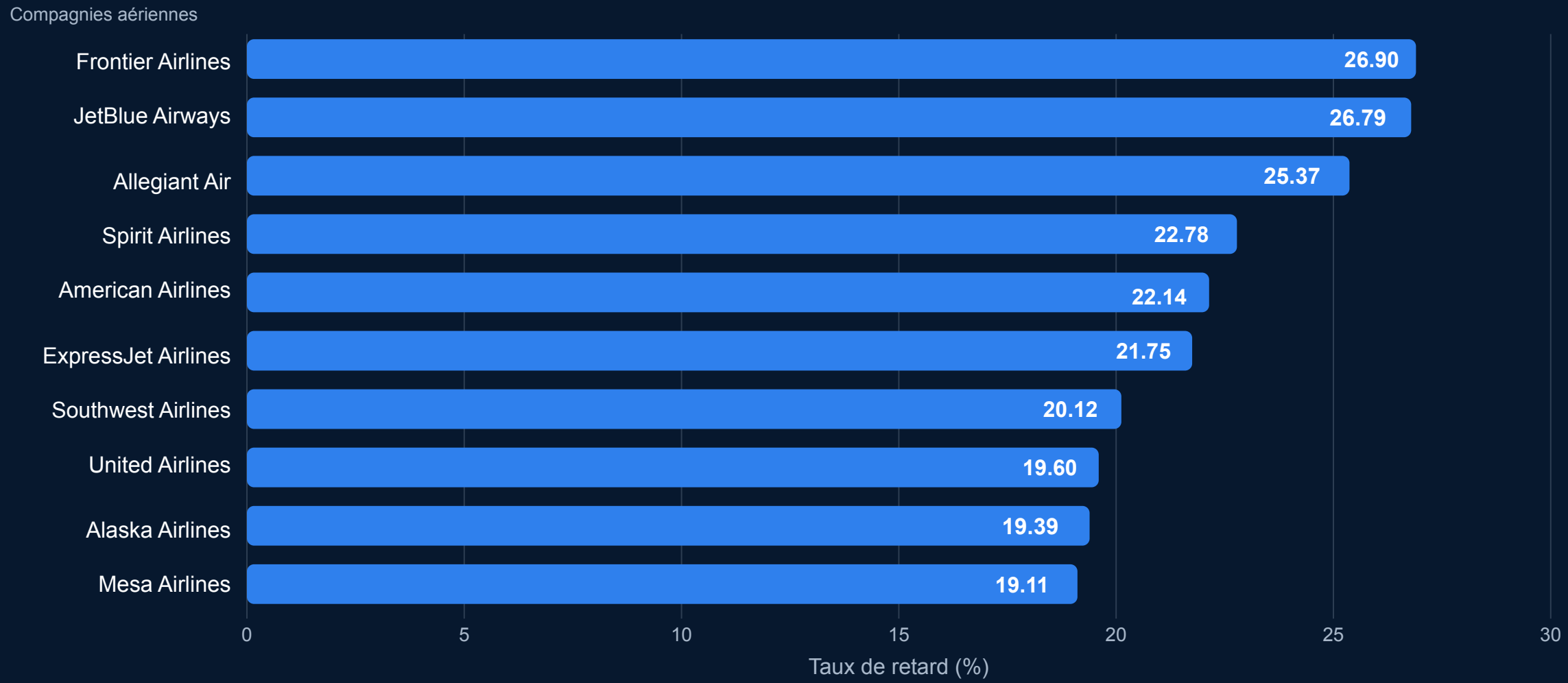
Dimensions (lignes insérées)

24 Dim_Time	2 738 Dim_Date_Flights	90 Dim_Date_Passengers
18 Dim_Airline	6 693 Dim_Airport	120 Dim_Weather (sample de test)

GRAPHIQUE POWER BI

Classement par taux de retard (top 10)

Taux = vols arrivés avec au moins 15 minutes de retard. Les vols annulés sont exclus.



Frontier Airlines, JetBlue Airways et Allegiant Air arrivent en tête : environ un vol opéré sur quatre est retardé d'au moins 15 minutes.

Difficultés techniques rencontrées

Pression mémoire

La machine initiale devait faire fonctionner Windows, SQL Server, WSL, Airflow et PySpark avec une mémoire limitée.

→ Ralentissement des tâches et processus

Charge SQL Server

Les écritures JDBC massives provoquaient une saturation des ressources et des transactions trop longues.

→ Réduction de la taille des transactions, davantage de partitions et parallélisme contrôlé.

Mise en place de l'infrastructure

Configuration de PySpark et du driver JDBC en local et trouver les bonnes valeurs pour le bon fonctionnement du projet

→ Lecture de documentation et multiples tests pour régler les divers problèmes

PROCHAINES ÉTAPES

Améliorations envisagées

01  **Enrichir le reporting**

Créer davantage de vues métier et de visualisations Power BI pour enrichir la visualisation de nos données

02  **Compléter la donnée météo**

Alimenter la base de donnée avec l'ensemble des appels API nécessaires afin d'analyser plus finement la nature des intempéries

03  **Poursuivre l'optimisation**

Réduire les temps d'exécution et la consommation mémoire, en particulier sur les chargements JDBC les plus volumineux.